Session 09: Docker & Containerization

DevOps Fundamentals

Rikkei Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

Lesson 01: Khác biệt giữa Virtual Machine và Container

Lesson 02: Viết Dockerfile dạng Single-stage build

Lesson 03: Viết Dockerfile dạng Multi-stage build

Lesson 04: Docker Networking, Docker Volumes

Lesson 05: Giới thiệu về Docker Compose

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

1

Khác biệt giữa Virtual Machine và Container

Khác biệt môi trường khi triển khai

Nguyên nhân ứng dụng chạy được ở máy phát triển nhưng lỗi trên server

```
TERMINAL

# trên máy của bạn
$ ./gradlew bootRun
Started Application in 3.2 seconds

# trên server của khách hàng
$ java -jar app.jar
Exception in thread "main"
  UnsupportedClassVersionError: Application
  has been compiled by a more recent version
  of the Java Runtime (class file 21.0),
  this version recognizes 17.0

$ java -version  → openjdk 17.0.9
```

Môi trường không bao giờ giống nhau

Khác phiên bản JDK, khác biến môi trường, khác thư viện hệ thống

Tài liệu cài đặt luôn lạc hậu

README mười bước, trong đó bước thứ tư đã đổi từ ba tháng trước

Không ai dựng lại được từ đầu

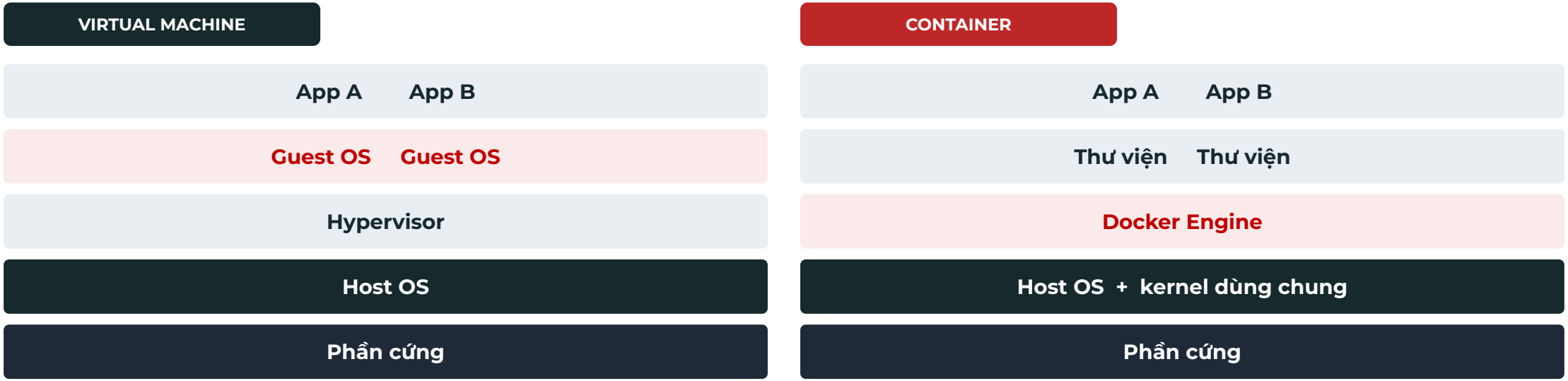
Người viết code nghỉ việc là không ai biết server cần đúng những gì

Câu trả lời của container: đóng gói cả ứng dụng và toàn bộ môi trường của nó vào một artifact duy nhất, rồi chạy đúng artifact đó ở mọi nơi.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Kiến trúc VM và Container

Cùng mục tiêu cô lập ứng dụng nhưng thực hiện ở hai tầng khác nhau



- Mỗi máy ảo mang theo cả một hệ điều hành
- Khởi động tính bằng chục giây đến vài phút
- Chiếm vài GB RAM và ổ đĩa ngay khi chưa làm gì

- Dùng chung kernel với máy chủ, không có Guest OS
- Khởi động dưới một giây, gần như tức thì
- Chỉ tốn tài nguyên khi tiến trình bên trong chạy

Container không phải "máy ảo nhẹ" — nó là một tiến trình bị cô lập bằng namespace và cgroup của Linux kernel.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

So sánh Virtual Machine và Container

Hai công nghệ bổ sung cho nhau: container thường chạy bên trong máy ảo

Tiêu chí	Virtual Machine	Container
Mức cô lập	Ảo hoá cả phần cứng, cô lập rất mạnh	Cô lập tiến trình bằng namespace
Hệ điều hành	Mỗi máy ảo có kernel riêng	Dùng chung kernel với host
Kích thước	Vài GB cho mỗi máy	Vài chục đến vài trăm MB
Thời gian khởi động	30 giây đến vài phút	Thường dưới một giây
Mật độ trên một host	Hàng chục máy ảo	Hàng trăm đến hàng nghìn container
Chạy hệ điều hành khác	Được — Windows trên máy Linux	Không — container Linux cần kernel Linux
Phù hợp nhất với	Cô lập tuyệt đối, chạy nhiều loại OS	Đóng gói và triển khai ứng dụng

Trong thực tế: EC2 hay Cloud Server của bạn là một máy ảo; Docker chạy bên trong nó. Hai công nghệ bổ sung cho nhau, không loại trừ.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Kiến trúc Docker Components

Docker CLI chỉ gửi yêu cầu; Docker Daemon là thành phần thực thi



BỐN LOẠI ĐỐI TƯỢNG DAEMON QUẢN LÝ

Image	Bản đóng gói chỉ đọc: ứng dụng cùng toàn bộ môi trường cần để chạy
Container	Một image đang chạy, kèm thêm một lớp ghi được của riêng nó
Volume	Vùng lưu trữ do Docker quản lý, sống độc lập với container
Network	Mạng ảo để các container gọi nhau bằng tên, không cần biết IP

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Phân biệt Image và Container

Image là bản thiết kế chỉ đọc; container là một thực thể đang chạy dựng từ bản thiết kế đó

MỘT IMAGE LÀ CHỒNG CÁC LỚP CHỈ ĐỌC

Lớp ghi được của container	mất khi container bị xoá
ENTRYPOINT java -jar	chỉ đọc
COPY app.jar	chỉ đọc
FROM eclipse-temurin:21-jre	chỉ đọc

Nhiều container chạy từ cùng một image sẽ chia sẻ các lớp bên dưới — vì thế container rất nhẹ

VÒNG ĐỜI

```
$ docker pull mysql:8
$ docker images          # xem image
$ docker run -d --name db mysql:8
$ docker ps -a           # cả cái đã tắt
$ docker stop db
$ docker rm db           # xoá container
$ docker rmi mysql:8     # xoá image

# image vẫn còn sau khi xoá container
# một image → nhiều container cùng lúc
```

Điểm mấu chốt: mọi dữ liệu ghi vào lớp writable sẽ mất cùng container. Đó chính là lý do phần 4 phải nói về volume.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

2

Viết Dockerfile dạng Single-stage build

Cấu trúc Dockerfile và cơ chế Layer

Docker cache theo từng lớp nên thứ tự các chỉ thị rất quan trọng

Lệnh	Ý nghĩa
FROM	Chọn image nền để bắt đầu
WORKDIR	Đặt thư mục làm việc cho các lệnh sau
COPY	Chép file từ build context vào image
RUN	Chạy lệnh lúc build, kết quả nằm trong lớp mới
ARG	Biến chỉ tồn tại lúc build, không còn khi container chạy
ENV	Biến lưu trong image: dùng ở lệnh build sau và khi chạy
EXPOSE	Ghi chú cổng ứng dụng nghe — không tự mở cổng
ENTRYPOINT	Lệnh chạy khi container khởi động
CMD	Tham số mặc định, có thể ghi đè khi docker run

Chọn ARG cho thông tin chỉ cần lúc build; chọn ENV khi ứng dụng cần đọc biến đó lúc chạy.

CACHE HOẠT ĐỘNG THẾ NÀO

FROM eclipse-temurin:21-jdk	**dùng cache**
COPY ./gradlew build.gradle ./	**dùng cache**
RUN ./gradlew dependencies	**dùng cache**
COPY src ./src	**build lại**
RUN ./gradlew clean build -x test --no-daemon	**build lại**
ENTRYPOINT ["java","-jar",...]	**build lại**

Sửa một dòng trong src/ → mọi lớp từ COPY src trở xuống phải dựng lại, các lớp trên vẫn dùng cache.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Single-stage build cho Spring Boot

Một Dockerfile duy nhất thực hiện cả biên dịch và chạy ứng dụng

Dockerfile

```
# một stage: vừa build vừa chạy
FROM eclipse-temurin:21-jdk

WORKDIR /app

# tách dependency ra để tận dụng cache
COPY .gradle/ .gradle
COPY ./gradlew build.gradle ./
RUN ./gradlew dependencies --no-daemon

COPY src ./src
RUN ./gradlew clean build -x test --no-daemon

EXPOSE 8080
ENTRYPOINT ["java","-jar","libs/app.jar"]
```

1. **FROM ... -jdk**
   Phải là JDK vì Maven biên dịch ngay trong image; bản JRE không đủ công cụ

2. **COPY pom trước COPY src**
   Dependency ít thay đổi nên tách riêng để Docker tái dùng lớp cache

3. **RUN ./gradlew clean build**
   Biên dịch trong môi trường đã chuẩn hoá, giảm đáng kể khác biệt giữa các máy build

4. **ENTRYPOINT dạng mảng (exec form)**
   Tiến trình ứng dụng nhận signal trực tiếp; shell form thêm một tiến trình trung gian nên dừng tử tế khó hơn

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Bộ lệnh build và chạy Container

Vòng lặp làm việc: build → run → logs → exec → chỉnh sửa → build lại

Lệnh	Làm gì	Ghi nhớ
docker build -t taskapi:1.0 .	Đọc Dockerfile và tạo image	Dấu chấm cuối là build context
docker images	Liệt kê image kèm kích thước	Cột SIZE là nơi thấy vấn đề
docker run -d -p 8080:8080 taskapi:1.0	Tạo và chạy một container	Thứ tự là host:container
docker ps · docker ps -a	Xem container đang chạy hoặc tất cả	Có -a mới thấy cái đã tắt
docker logs -f <id>	Đọc log ứng dụng trong container	Nơi đầu tiên cần xem khi lỗi
docker exec -it <id> sh	Vào bên trong container đang chạy	Kiểm tra file và biến môi trường
docker stop · rm · rmi	Dừng, xoá container, xoá image	Phải stop trước khi rm

.dockerignore quan trọng như .gitignore: không có nó thì target/, .git/ và toàn bộ file rác bị gửi vào build context, làm build chậm và image phình.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

3

Viết Dockerfile dạng Multi-stage build

Nguyên lý Multi-stage build

Stage đầu có đủ công cụ để biên dịch; image cuối chỉ nhận đúng file .jar



Docker chỉ giữ lại stage cuối cùng — toàn bộ stage 1 bị loại khỏi image kết quả

Không còn công cụ build

JDK, Gradle, cache dependency và source code đều ở lại stage 1

Bề mặt tấn công nhỏ hơn

Ít gói phần mềm trong image nghĩa là ít lỗ hổng phải vá

Tải và khởi động nhanh hơn

Image nhẹ thì pull nhanh, scale nhanh, tiết kiệm băng thông registry

Quy tắc: mọi image lên production đều phải là multi-stage. Không có lý do nào để đẩy JDK và source code lên server.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Multi-stage build cho Spring Boot

Cùng ứng dụng Spring Boot, viết lại thành hai stage cho chuẩn production

Dockerfile

```
# ---------- stage 1: build ----------
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /build
COPY .gradle/ .gradle
COPY ./gradlew build.gradle ./
RUN ./gradlew dependencies --no-daemon
COPY src ./src
RUN ./gradlew clean build -x test --no-daemon

# ---------- stage 2: runtime ----------
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=builder /build/libs/*.jar app.jar
USER app
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

1. **AS builder**
   Đặt tên cho stage để stage sau tham chiếu tới được

2. **COPY --from=builder**
   Chỉ nhặt đúng file .jar; JDK, Gradle và source ở lại stage 1

3. **jre-alpine**
   Runtime chỉ cần JRE, bản alpine nhỏ hơn bản mặc định vài trăm MB

4. **USER app**
   Không chạy bằng root trong container để giảm thiệt hại khi bị khai thác

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Tối ưu kích thước Image

Cùng một ứng dụng, kích thước image chênh gần bốn lần theo cách viết Dockerfile

SINGLE-STAGE

712 MB

JDK + Maven + cache + source + jar

MULTI-STAGE

186 MB

JRE alpine + đúng một file app.jar

Kỹ thuật	Cách làm	Được gì
Multi-stage build	Chỉ COPY --from artifact cuối cùng	**Giảm khoảng 70% kích thước**
Base image nhỏ	jre-alpine thay cho jdk đầy đủ	**Bớt khoảng 300 MB**
.dockerignore	Loại build/, .git/, .idea/ khỏi context	**Build nhanh, context gọn**
Layer caching	COPY build.gradle trước khi COPY src	**Build lại nhanh hơn nhiều lần**
USER không phải root	addgroup + adduser rồi USER app	**Giảm hậu quả khi container bị chiếm**

Đo trước khi tối ưu: docker images và docker history <image> cho biết đúng lớp nào đang chiếm chỗ.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

4

Docker Networking &

Docker Volumes

Các chế độ Docker Networking

Chế độ mặc định là bridge; phần lớn dự án chỉ cần một bridge riêng

bridge

MẶC ĐỊNH

Docker tạo một mạng ảo riêng, mỗi container có IP nội bộ và ra Internet qua NAT

Dùng cho gần như mọi dự án

host

KHÔNG CÁCH LY

Container dùng thẳng network stack của máy chủ, không map cổng nữa

Khi cần hiệu năng mạng tối đa

none

CÔ LẬP HOÀN TOÀN

Chỉ có loopback, không kết nối được ra ngoài hay tới container khác

Cho tác vụ xử lý dữ liệu offline

```
$ docker network create app-net
$ docker network ls
NETWORK ID   NAME      DRIVER   SCOPE
a1b2c3d4     app-net   bridge   local
b2c3d4e5     bridge    bridge   local
```

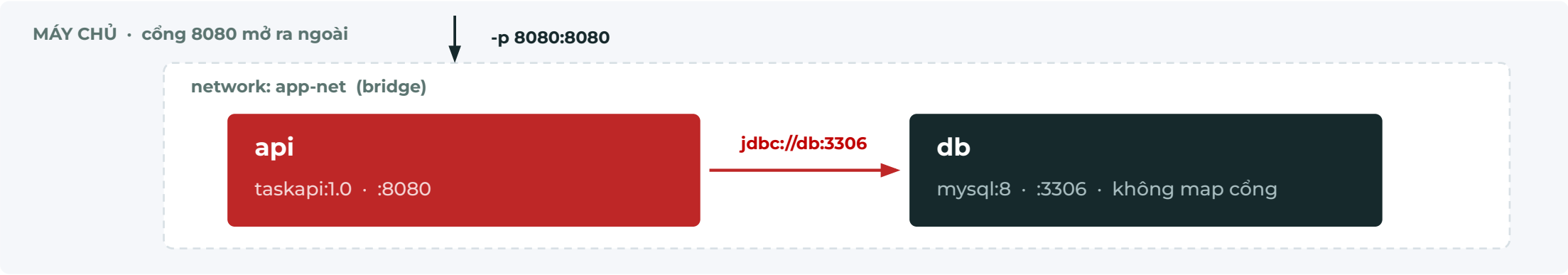
DNS nội bộ chỉ có ở bridge tự tạo

Trong một network do bạn tạo, container gọi nhau bằng chính tên container. Mạng bridge mặc định của Docker thì không có tính năng này — đó là lý do luôn tạo network riêng.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Kết nối Container qua Docker Network

Tên container đóng vai trò hostname, không cần tra địa chỉ IP



TERMINAL

```
$ docker network create app-net
$ docker run -d --name db --network app-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=taskdb mysql:8
$ docker run -d --name api --network app-net \
  -p 8080:8080 \
  -e SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/taskdb \
  taskapi:1.0
```

Tên container là hostname
jdbc:mysql://db:3306 — Docker tự phân giải, không cần biết IP

Chỉ map cổng khi thật cần
db không cần -p; chỉ api mới cần cổng 8080 ra máy chủ

localhost trong container là chính nó
Đây là lỗi phổ biến nhất của người mới dùng Docker

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Tính tạm thời của dữ liệu Container

Lớp writable bị xoá cùng container, kéo theo toàn bộ dữ liệu bên trong

THỬ NGHIỆM

```
$ docker run -d --name db mysql:8
# ... tạo bảng tasks, thêm 10.000 dòng ...

$ docker rm -f db
$ docker run -d --name db mysql:8

mysql> SELECT count(*) FROM tasks;
ERROR 1146: Table 'taskdb.tasks' doesn't exist

# lớp writable của container cũ đã bị xoá
# cùng với toàn bộ dữ liệu bên trong nó
```

Volume

Docker quản lý, nằm trong /var/lib/docker/volumes

→ **Dữ liệu database ở production**

Bind mount

Gắn thẳng một thư mục của máy chủ vào container

→ **Đồng bộ code khi đang phát triển**

tmpfs

Chỉ nằm trong RAM, mất khi container dừng

→ **Dữ liệu tạm và thông tin bí mật**

Nguyên tắc: container phải luôn có thể bị xoá và dựng lại mà không mất gì. Mọi dữ liệu cần sống lâu đều phải nằm ngoài container.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Lưu trữ bền vững với Docker Volume

Gắn volume có tên vào thư mục dữ liệu của MySQL

```
TERMINAL
$ docker volume create mysql-data

$ docker run -d --name db --network app-net \
    -v mysql-data:/var/lib/mysql \
    -e MYSQL_ROOT_PASSWORD=secret mysql:8

$ docker volume ls
$ docker volume inspect mysql-data

# xoá container rồi dựng lại — dữ liệu vẫn còn
$ docker rm -f db
$ docker run -d ... -v mysql-data:/var/lib/mysql ...

# sao lưu volume ra file tar
$ docker run --rm -v mysql-data:/data \
    -v $PWD:/backup alpine tar czf /backup/db.tgz /data
```

1. **-v <volume>:<đường dẫn>**
   Bên trái là tên volume, bên phải là thư mục dữ liệu trong container

2. **Mỗi image có đường dẫn riêng**
   MySQL dùng /var/lib/mysql · PostgreSQL dùng /var/lib/postgresql/data

3. **docker volume prune xoá thật**
   Không có thùng rác nào — đọc kỹ danh sách trước khi xác nhận

4. **Volume không phải bản backup**
   Vẫn cần dump định kỳ ra ngoài máy chủ hoặc lên lưu trữ đám mây

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

5

Giới thiệu về Docker Compose

Vai trò của Docker Compose

Cùng dựng một hệ thống App + Database với hai cách tiếp cận khác nhau

GÕ TAY MỖI LẦN

```
$ docker network create app-net
$ docker volume create mysql-data
$ docker run -d --name db --network ... \
    -v mysql-data:/var/lib/mysql -e ...
$ docker build -t taskapi:1.0 .
$ docker run -d --name api --network ... \
    -p 8080:8080 -e SPRING_DATASOURCE...
```

KHAI BÁO MỘT LẦN

```
$ docker compose up -d

[+] Running 4/4
 ✔ Network taskapi_default   Created
 ✔ Volume  mysql-data        Created
 ✔ Container taskapi-db-1    Healthy
 ✔ Container taskapi-api-1   Started
```

Hệ thống được ghi thành tài liệu

File YAML nằm trong repo, ai clone về cũng dựng lên được y hệt

Không còn tham số bị quên

Network, volume, biến môi trường và thứ tự khởi động đều nằm trong file

Một lệnh dựng, một lệnh dọn

docker compose up -d và docker compose down cho toàn bộ hệ thống

Compose không phải công cụ production quy mô lớn — nhưng cho môi trường phát triển và server đơn lẻ thì rất đủ.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Cấu trúc file docker-compose.yml

Toàn bộ hệ thống của các phần trước được khai báo trong một file

docker-compose.yml

```yaml
services:
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/taskdb
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    depends_on:
      db: { condition: service_healthy }
    restart: unless-stopped
  db:
    image: mysql:8
    volumes: [ mysql-data:/var/lib/mysql ]
    healthcheck:
      test: ["CMD","mysqladmin","ping"]
volumes:
  mysql-data:
```

1. **services**
   Mỗi service là một container; tên service chính là hostname trong mạng nội bộ

2. **depends_on + healthcheck**
   Chờ database sẵn sàng thật, không chỉ chờ container vừa được tạo

3. **${DB_PASSWORD}**
   Giá trị lấy từ file .env — không bao giờ viết mật khẩu vào compose file

4. **volumes ở cấp trên cùng**
   Khai báo volume có tên để dữ liệu sống qua cả docker compose down

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Bộ lệnh Docker Compose

Các lệnh được gọi theo tên service khai báo trong file YAML

Lệnh	Làm gì	Ghi nhớ
docker compose up -d	Tạo network, volume và chạy mọi service	Thêm --build để buộc build lại
docker compose down	Dừng và xoá container cùng network	Volume vẫn còn; -v mới xoá volume
docker compose ps	Trạng thái từng service	Cột STATUS hiển thị healthy
docker compose logs -f api	Xem log của đúng một service	Bỏ tên service để xem tất cả
docker compose exec api sh	Vào bên trong một service đang chạy	Gọi theo tên service
docker compose build --no-cache	Build lại từ đầu, bỏ toàn bộ cache	Khi Dockerfile đổi mà cache cũ
docker compose config	In cấu hình sau khi thay biến	Kiểm tra .env đã được nạp chưa

docker compose hay docker-compose? Bản có dấu cách là v2 tích hợp trong Docker CLI. Bản có gạch nối là v1 viết bằng Python và đã ngừng hỗ trợ.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

TỔNG KẾT

- Phân biệt Virtual Machine với Container và hiểu kiến trúc Docker
- Viết Dockerfile single-stage rồi tối ưu bằng multi-stage build
- Nối container qua network và giữ dữ liệu bền vững bằng volume
- Điều phối toàn bộ hệ thống App + Database bằng Docker Compose

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN